

Understanding CNN Internals

Convolution, ReLU, Pooling & Classification

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# Function to display matrix graphically
# -----
def Marvellous_Display(matrix, title):
    plt.figure(figsize=(4, 4))
    plt.imshow(matrix, cmap='gray', interpolation='nearest')
    plt.title(title)
    plt.colorbar()

    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            plt.text(j, i, f"{matrix[i][j]:.1f}",
                    ha='center', va='center',
                    color='red', fontsize=12)

    plt.show()

# -----
# Function to print separator
# -----
def Marvellous_Line():
    print("\n" + "-" * 60)

# -----
# Function : Convolution with detailed calculations
# -----
def Marvellous_Convolution(image, kernel):

    rows, cols = image.shape
    krows, kcols = kernel.shape

    output_rows = rows - krows + 1
    output_cols = cols - kcols + 1

    output = np.zeros((output_rows, output_cols))

    Marvellous_Line()
    print("STEP 1 : CONVOLUTION LAYER")
    Marvellous_Line()

    for i in range(output_rows):
        for j in range(output_cols):

            region = image[i:i+krows, j:j+kcols]
            multiplication = region * kernel
            result = np.sum(multiplication)

            output[i][j] = result

    print(f"\nRegion position -> Row:{i} Column:{j}")
    print("\nSelected Region:")
```

```

print(region)

print("\nKernel:")
print(kernel)

print("\nRegion * Kernel:")
print(multiplication)

print("\nSum of all values =", result)

print("\nFinal Convolution Output:")
print(output)

return output

# -----
# Function : ReLU with detailed calculations
# -----
def Marvellous_ReLU(data):

    Marvellous_Line()
    print("STEP 2 : RELU ACTIVATION")
    Marvellous_Line()

    output = np.maximum(0, data)

    print("\nInput to ReLU:")
    print(data)

    print("\nRule : ReLU(x) = max(0, x)")
    print("\nOutput after ReLU:")
    print(output)

    return output

# -----
# Function : Max Pooling with detailed calculations
# -----
def Marvellous_Pooling(data):

    rows, cols = data.shape

    output_rows = rows // 2
    output_cols = cols // 2

    output = np.zeros((output_rows, output_cols))

    Marvellous_Line()
    print("STEP 3 : MAX POOLING")
    Marvellous_Line()

    r = 0
    for i in range(0, rows, 2):
        c = 0
        for j in range(0, cols, 2):

            block = data[i:i+2, j:j+2]

            # Skip incomplete blocks if any

```

```

if block.shape != (2, 2):
    continue
max_value = np.max(block)
output[r][c] = max_value

print(f"\nPooling Block position -> Row:{r} Column:{c}")
print("\nSelected 2x2 Block:")
print(block)

print("\nMaximum value selected =", max_value)

c += 1
r += 1

print("\nFinal Pooling Output:")
print(output)

return output

# -----
# Function : Flatten with detailed output
# -----
def Marvellous_Flatten(data):

    Marvellous_Line()
    print("STEP 4 : FLATTEN LAYER")
    Marvellous_Line()

    flat = data.flatten()

    print("\nInput to Flatten:")
    print(data)

    print("\nFlattened Output:")
    print(flat)

    return flat

# -----
# Function : Fully Connected Layer with detailed calculation
# -----
def Marvellous_FC(flat_data):

    Marvellous_Line()
    print("STEP 5 : FULLY CONNECTED LAYER")
    Marvellous_Line()

    # manual weights
    weights = np.array([1, 1, 1, 1], dtype=float)
    bias = 0.0

    print("\nFlatten Input:")
    print(flat_data)

    print("\nWeights:")
    print(weights)

    print("\nBias:")
    print(bias)

```

```

multiplication = flat_data * weights
result = np.sum(multiplication) + bias

print("\nInput * Weights:")
print(multiplication)

print("\nSum =", np.sum(multiplication))
print("Final Output after adding bias =", result)

return result

# -----
# Main Function
# -----
def Marvellous_CNN():

    print("Choose Input Image")
    print("1 : Vertical Line")
    print("2 : Horizontal Line")

    choice = int(input("Enter your choice : "))

    # -----
    # Input Image
    # -----
    if choice == 1:
        image = np.array([
            [0, 0, 1, 0, 0],
            [0, 0, 1, 0, 0],
            [0, 0, 1, 0, 0],
            [0, 0, 1, 0, 0],
            [0, 0, 1, 0, 0],
            [0, 0, 1, 0, 0]
        ], dtype=float)
        actual = "Vertical Line"

    else:
        image = np.array([
            [0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0],
            [1, 1, 1, 1, 1],
            [0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0]
        ], dtype=float)
        actual = "Horizontal Line"

    Marvellous_Line()
    print("INPUT IMAGE")
    Marvellous_Line()
    print("\nActual Input =", actual)
    print("\nInput Matrix:")
    print(image)
    Marvellous_Display(image, "Input Image")

    # -----
    # Kernel for Vertical Feature Detection
    # -----
    kernel = np.array([
        [-1, 1, -1],
        [-1, 1, -1],

```

```

[-1, 1, -1]
], dtype=float)

Marvellous_Line()
print("KERNEL")
Marvellous_Line()
print("\nKernel used to detect vertical pattern:")
print(kernel)
Marvellous_Display(kernel, "Vertical Detection Kernel")

# -----
# Step 1 : Convolution
# -----
conv = Marvellous_Convolution(image, kernel)
Marvellous_Display(conv, "Convolution Output")

# -----
# Step 2 : ReLU
# -----
relu = Marvellous_ReLU(conv)
Marvellous_Display(relu, "ReLU Output")

# -----
# Step 3 : Pooling
# -----
pool = Marvellous_Pooling(relu)
Marvellous_Display(pool, "Pooling Output")

# -----
# Step 4 : Flatten
# -----
flat = Marvellous_Flatten(pool)

# -----
# Step 5 : Fully Connected Layer
# -----
score = Marvellous_FC(flat)

# -----
# Final Prediction
# -----
Marvellous_Line()
print("STEP 6 : FINAL PREDICTION")
Marvellous_Line()

print("\nFinal Score =", score)

if score > 0:
    prediction = "Vertical Line"
else:
    prediction = "Horizontal Line"

print("\nPredicted Output =", prediction)
print("Actual Input =", actual)

Marvellous_CNN()

```

This program demonstrates how a **Convolutional Neural Network (CNN)** works in a very simple way.

In this program, we take a simple image made of numbers and perform:

1. Input Image
2. Convolution Layer
3. ReLU Activation
4. Pooling Layer
5. Flatten Layer
6. Fully Connected Layer
7. Final Prediction

CNN :

CNN stands for **Convolutional Neural Network**.

It is mainly used for:

- Image Classification
- Face Recognition
- Object Detection
- Medical Image Analysis
- Handwritten Digit Recognition

CNN is excellent at detecting:

- Edges
- Lines
- Shapes
- Patterns
- Textures

We are classifying image into two classes:

- **Vertical Line**
- **Horizontal Line**

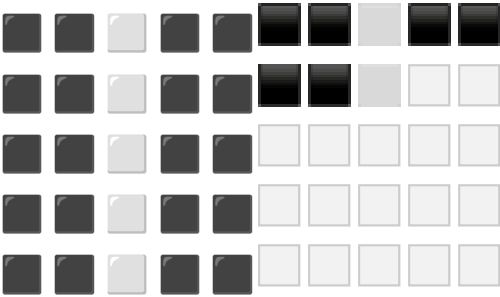
This is called **Binary Classification**.

Input Images

A) Vertical Line Image

```
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
```

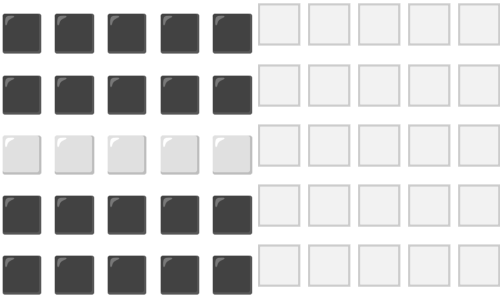
Visual Form:



B) Horizontal Line Image

0 0 0 0 0
0 0 0 0 0
1 1 1 1 1
0 0 0 0 0
0 0 0 0 0

Visual Form:



Meaning of Pixel Values

- 0 = Black Pixel
- 1 = White Pixel

CNN Flow

Input Image
↓
Convolution
↓
ReLU
↓
Pooling
↓
Flatten
↓
Fully Connected Layer
↓
Prediction

Libraries Used

```
import numpy as np
import matplotlib.pyplot as plt
```

Explanation

numpy

Used for:

- Matrix operations
- Multiplication
- Summation
- Arrays

matplotlib

Used for:

- Showing image
- Graphical output

Display Function

```
def Marvellous_Display(matrix, title):
```

Used to display matrix as image.

Convolution Layer

Kernel Used

-1 1 -1
-1 1 -1
-1 1 -1

This kernel detects **Vertical Line**.

This Kernel Detects Vertical Line :

Center column has positive values:

1
1
1

Side columns have negative values:

-1 -1
-1 -1
-1 -1

So if white pixels are in center, output becomes high positive.

Output Size After Convolution

Input Image = 5 x 5

Kernel = 3 x 3

Formula:

Output Rows = $5 - 3 + 1 = 3$

Output Cols = $5 - 3 + 1 = 3$

So output size = **3 x 3**

Convolution Formula

$\text{Output}(i,j) = \sum(\text{ImageRegion} \times \text{Kernel})$

Detailed Convolution Example

Region 1

Selected Region:

0 0 1
0 0 1
0 0 1

Kernel:

-1 1 -1
-1 1 -1
-1 1 -1

Multiplication:

0×-1 0×1 1×-1
0×-1 0×1 1×-1
0×-1 0×1 1×-1

Result:

0 0 -1
0 0 -1
0 0 -1

Sum:

-3

Region 2

Selected Region:

0 1 0
0 1 0
0 1 0

Multiplication:

0×-1 1×1 0×-1
0×-1 1×1 0×-1
0×-1 1×1 0×-1

Result:

0 1 0
0 1 0
0 1 0

Sum:

3

Region 3

Selected Region:

1 0 0
1 0 0
1 0 0

Result Sum:

-3

Final Convolution Output

-3 3 -3
-3 3 -3
-3 3 -3

ReLU Activation

Formula:

$\text{ReLU}(x) = \max(0, x)$

Meaning:

- Negative $\rightarrow 0$
- Positive $\rightarrow$ Same value

Example

Input:

-3 3 -3
-3 3 -3
-3 3 -3

Output:

0 3 0
0 3 0
0 3 0

ReLU Used for :

- Removes negative values
- Keeps useful features
- Adds non-linearity
- Faster training in real CNN

Pooling Layer

We use **Max Pooling**

It selects maximum value from 2x2 block.

Example

Input:

```
0  3
0  3
```

Maximum:

```
3
```

Final Pooling Output

```
3
```

Pooling Used to :

- Reduces size
- Reduces calculations
- Keeps important features
- Faster processing

Flatten Layer

Converts matrix into vector.

Example:

```
3
```

Becomes:

```
[3]
```

Fully Connected Layer

Formula:

Output= $\Sigma(\text{Input} \times \text{Weight}) + \text{Bias}$

Suppose:

Input:

[3]

Weight:

[1]

Bias:

0

Calculation:

$(3 \times 1) + 0 = 3$

Final Score = **3**

Final Prediction Logic

```
if score > 0:  
    Vertical Line  
else:  
    Horizontal Line
```

Full Result for Vertical Line

Input

```
0 0 1 0 0  
0 0 1 0 0  
0 0 1 0 0  
0 0 1 0 0  
0 0 1 0 0
```

Convolution

```
-3 3 -3  
-3 3 -3  
-3 3 -3
```

ReLU

```
0 3 0  
0 3 0
```

0 3 0

Pooling

3

Flatten

[3]

FC Output

3

Prediction

Vertical Line

Result for Horizontal Line

Input:

```
0 0 0 0 0
0 0 0 0 0
1 1 1 1 1
0 0 0 0 0
0 0 0 0 0
```

Since kernel is for vertical detection:

- weak response
- low values
- output near zero

Prediction:

Horizontal Line

Important Terms

Image

Matrix of pixels.

Pixel

Smallest part of image.

Kernel / Filter

Small matrix used to detect pattern.

Feature Map

Output of convolution.

Activation Function

Function after convolution.

Pooling

Reduce feature map size.

Flatten

Convert matrix to vector.

Fully Connected Layer

Final decision layer.